
Samsung HBM-PIM Execution Flow

1st, Mar. 2026 Minsung Kim

***Intelligent System
Laboratory***

Samsung HBM-PIM Hardware

□ HBM2-PIM Die

HBM2 spec

→ (2M x 16 x 64)bit

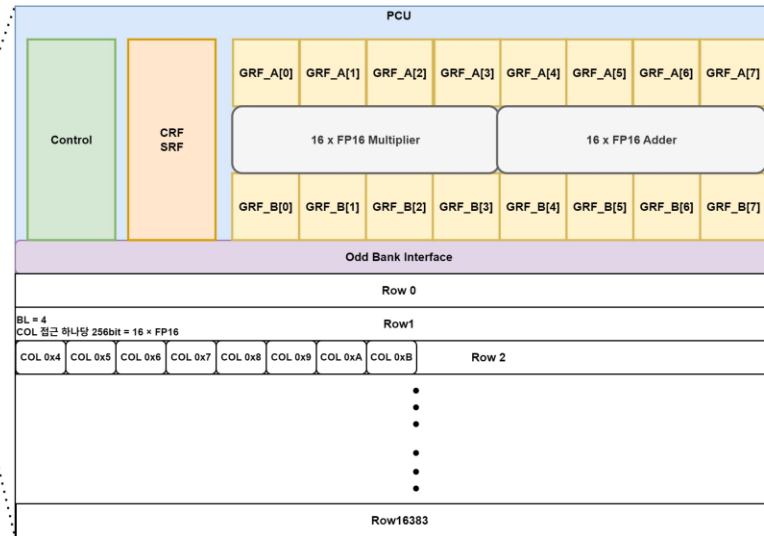
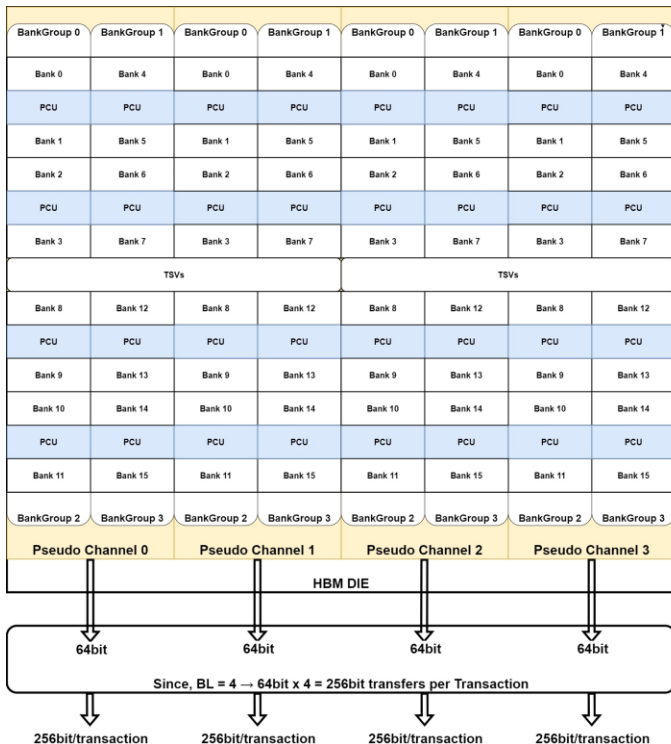
→ 2Gb

(per Pseudo-Channel)

NUM_BANK_GROUPS=4
NUM_BANKS=16
NUM_COLS=128
NUM_ROWS=16384
NUM_PIM_BLOCKS=8
DEVICE_WIDTH=64
BL=4
RL=20
WL=8

- Pseudo channel 당 256bit/transaction
- GRF 1개는 16 x FP16 (=256bit)을 저장
- GRF_A 8개, GRF_B 8개로 구성
- CRF(command register file), SRF(scalar register file)
- SIMD FP16 Multiplier & Adder

- HBM-PIM paper(ISCAS 2021)의 figure 5.에 제시된 MAC연산 example
- simulator에서 이러한 operation이 doPIMBlock()에 구현되어 있음



- AAM mode에서 GRF_A/GRF_B의 index가 어떻게 정해지는지

Row : 14bit

Column : 7bit → Since, BL = 4 (2bit), only 5bit(7bit-2bit) is needed to access column

GEMV example in paper

matrix A[4x16], vector X[16] → Y[8]

(j=0~15)

GRF_B[0] = Z A[0,j] × X[j]

GRF_B[0] = Z A[1,j] × X[j]

GRF_B[0] = Z A[2,j] × X[j]

GRF_B[0] = Z A[3,j] × X[j]

GRF_B[0] = Z A[4,j] × X[j]

GRF_B[0] = Z A[5,j] × X[j]

GRF_B[0] = Z A[6,j] × X[j]

GRF_B[0] = Z A[7,j] × X[j]

GRF_B[0] = Z A[8,j] × X[j]

GRF_B[0] = Z A[9,j] × X[j]

GRF_B[0] = Z A[10,j] × X[j]

GRF_B[0] = Z A[11,j] × X[j]

GRF_B[0] = Z A[12,j] × X[j]

GRF_B[0] = Z A[13,j] × X[j]

GRF_B[0] = Z A[14,j] × X[j]

GRF_B[0] = Z A[15,j] × X[j]

GRF_B[0] = Z A[16,j] × X[j]

GRF_B[0] = Z A[17,j] × X[j]

GRF_B[0] = Z A[18,j] × X[j]

GRF_B[0] = Z A[19,j] × X[j]

GRF_B[0] = Z A[20,j] × X[j]

GRF_B[0] = Z A[21,j] × X[j]

GRF_B[0] = Z A[22,j] × X[j]

GRF_B[0] = Z A[23,j] × X[j]

GRF_B[0] = Z A[24,j] × X[j]

GRF_B[0] = Z A[25,j] × X[j]

GRF_B[0] = Z A[26,j] × X[j]

GRF_B[0] = Z A[27,j] × X[j]

GRF_B[0] = Z A[28,j] × X[j]

GRF_B[0] = Z A[29,j] × X[j]

GRF_B[0] = Z A[30,j] × X[j]

GRF_B[0] = Z A[31,j] × X[j]

GRF_B[0] = Z A[32,j] × X[j]

GRF_B[0] = Z A[33,j] × X[j]

GRF_B[0] = Z A[34,j] × X[j]

GRF_B[0] = Z A[35,j] × X[j]

GRF_B[0] = Z A[36,j] × X[j]

GRF_B[0] = Z A[37,j] × X[j]

GRF_B[0] = Z A[38,j] × X[j]

GRF_B[0] = Z A[39,j] × X[j]

GRF_B[0] = Z A[40,j] × X[j]

GRF_B[0] = Z A[41,j] × X[j]

GRF_B[0] = Z A[42,j] × X[j]

GRF_B[0] = Z A[43,j] × X[j]

GRF_B[0] = Z A[44,j] × X[j]

GRF_B[0] = Z A[45,j] × X[j]

GRF_B[0] = Z A[46,j] × X[j]

GRF_B[0] = Z A[47,j] × X[j]

GRF_B[0] = Z A[48,j] × X[j]

GRF_B[0] = Z A[49,j] × X[j]

GRF_B[0] = Z A[50,j] × X[j]

GRF_B[0] = Z A[51,j] × X[j]

GRF_B[0] = Z A[52,j] × X[j]

GRF_B[0] = Z A[53,j] × X[j]

GRF_B[0] = Z A[54,j] × X[j]

GRF_B[0] = Z A[55,j] × X[j]

GRF_B[0] = Z A[56,j] × X[j]

GRF_B[0] = Z A[57,j] × X[j]

GRF_B[0] = Z A[58,j] × X[j]

GRF_B[0] = Z A[59,j] × X[j]

GRF_B[0] = Z A[60,j] × X[j]

GRF_B[0] = Z A[61,j] × X[j]

GRF_B[0] = Z A[62,j] × X[j]

GRF_B[0] = Z A[63,j] × X[j]

GRF_B[0] = Z A[64,j] × X[j]

GRF_B[0] = Z A[65,j] × X[j]

GRF_B[0] = Z A[66,j] × X[j]

GRF_B[0] = Z A[67,j] × X[j]

GRF_B[0] = Z A[68,j] × X[j]

GRF_B[0] = Z A[69,j] × X[j]

GRF_B[0] = Z A[70,j] × X[j]

GRF_B[0] = Z A[71,j] × X[j]

GRF_B[0] = Z A[72,j] × X[j]

GRF_B[0] = Z A[73,j] × X[j]

GRF_B[0] = Z A[74,j] × X[j]

GRF_B[0] = Z A[75,j] × X[j]

GRF_B[0] = Z A[76,j] × X[j]

GRF_B[0] = Z A[77,j] × X[j]

GRF_B[0] = Z A[78,j] × X[j]

GRF_B[0] = Z A[79,j] × X[j]

GRF_B[0] = Z A[80,j] × X[j]

GRF_B[0] = Z A[81,j] × X[j]

GRF_B[0] = Z A[82,j] × X[j]

GRF_B[0] = Z A[83,j] × X[j]

GRF_B[0] = Z A[84,j] × X[j]

GRF_B[0] = Z A[85,j] × X[j]

GRF_B[0] = Z A[86,j] × X[j]

GRF_B[0] = Z A[87,j] × X[j]

GRF_B[0] = Z A[88,j] × X[j]

GRF_B[0] = Z A[89,j] × X[j]

GRF_B[0] = Z A[90,j] × X[j]

GRF_B[0] = Z A[91,j] × X[j]

GRF_B[0] = Z A[92,j] × X[j]

GRF_B[0] = Z A[93,j] × X[j]

GRF_B[0] = Z A[94,j] × X[j]

GRF_B[0] = Z A[95,j] × X[j]

GRF_B[0] = Z A[96,j] × X[j]

GRF_B[0] = Z A[97,j] × X[j]

GRF_B[0] = Z A[98,j] × X[j]

GRF_B[0] = Z A[99,j] × X[j]

GRF_B[0] = Z A[100,j] × X[j]

GRF_B[0] = Z A[101,j] × X[j]

GRF_B[0] = Z A[102,j] × X[j]

GRF_B[0] = Z A[103,j] × X[j]

GRF_B[0] = Z A[104,j] × X[j]

GRF_B[0] = Z A[105,j] × X[j]

GRF_B[0] = Z A[106,j] × X[j]

GRF_B[0] = Z A[107,j] × X[j]

GRF_B[0] = Z A[108,j] × X[j]

GRF_B[0] = Z A[109,j] × X[j]

GRF_B[0] = Z A[110,j] × X[j]

GRF_B[0] = Z A[111,j] × X[j]

GRF_B[0] = Z A[112,j] × X[j]

GRF_B[0] = Z A[113,j] × X[j]

GRF_B[0] = Z A[114,j] × X[j]

GRF_B[0] = Z A[115,j] × X[j]

GRF_B[0] = Z A[116,j] × X[j]

GRF_B[0] = Z A[117,j] × X[j]

GRF_B[0] = Z A[118,j] × X[j]

GRF_B[0] = Z A[119,j] × X[j]

GRF_B[0] = Z A[120,j] × X[j]

GRF_B[0] = Z A[121,j] × X[j]

GRF_B[0] = Z A[122,j] × X[j]

GRF_B[0] = Z A[123,j] × X[j]

GRF_B[0] = Z A[124,j] × X[j]

GRF_B[0] = Z A[125,j] × X[j]

GRF_B[0] = Z A[126,j] × X[j]

GRF_B[0] = Z A[127,j] × X[j]

GRF_B[0] = Z A[128,j] × X[j]

GRF_B[0] = Z A[129,j] × X[j]

GRF_B[0] = Z A[130,j] × X[j]

GRF_B[0] = Z A[131,j] × X[j]

GRF_B[0] = Z A[132,j] × X[j]

GRF_B[0] = Z A[133,j] × X[j]

GRF_B[0] = Z A[134,j] × X[j]

GRF_B[0] = Z A[135,j] × X[j]

GRF_B[0] = Z A[136,j] × X[j]

GRF_B[0] = Z A[137,j] × X[j]

GRF_B[0] = Z A[138,j] × X[j]

GRF_B[0] = Z A[139,j] × X[j]

GRF_B[0] = Z A[140,j] × X[j]

GRF_B[0] = Z A[141,j] × X[j]

GRF_B[0] = Z A[142,j] × X[j]

GRF_B[0] = Z A[143,j] × X[j]

GRF_B[0] = Z A[144,j] × X[j]

GRF_B[0] = Z A[145,j] × X[j]

GRF_B[0] = Z A[146,j] × X[j]

GRF_B[0] = Z A[147,j] × X[j]

GRF_B[0] = Z A[148,j] × X[j]

GRF_B[0] = Z A[149,j] × X[j]

GRF_B[0] = Z A[150,j] × X[j]

GRF_B[0] = Z A[151,j] × X[j]

GRF_B[0] = Z A[152,j] × X[j]

GRF_B[0] = Z A[153,j] × X[j]

GRF_B[0] = Z A[154,j] × X[j]

GRF_B[0] = Z A[155,j] × X[j]

GRF_B[0] = Z A[156,j] × X[j]

GRF_B[0] = Z A[157,j] × X[j]

GRF_B[0] = Z A[158,j] × X[j]

GRF_B[0] = Z A[159,j] × X[j]

GRF_B[0] = Z A[160,j] × X[j]

GRF_B[0] = Z A[161,j] × X[j]

GRF_B[0] = Z A[162,j] × X[j]

GRF_B[0] = Z A[163,j] × X[j]

GRF_B[0] = Z A[164,j] × X[j]

GRF_B[0] = Z A[165,j] × X[j]

GRF_B[0] = Z A[166,j] × X[j]

GRF_B[0] = Z A[167,j] × X[j]

GRF_B[0] = Z A[168,j] × X[j]

GRF_B[0] = Z A[169,j] × X[j]

GRF_B[0] = Z A[170,j] × X[j]

GRF_B[0] = Z A[171,j] × X[j]

GRF_B[0] = Z A[172,j] × X[j]

GRF_B[0] = Z A[173,j] × X[j]

GRF_B[0] = Z A[174,j] × X[j]

GRF_B[0] = Z A[175,j] × X[j]

GRF_B[0] = Z A[176,j] × X[j]

GRF_B[0] = Z A[177,j] × X[j]

GRF_B[0] = Z A[178,j] × X[j]

GRF_B[0] = Z A[179,j] × X[j]

GRF_B[0] = Z A[180,j] × X[j]

GRF_B[0] = Z A[181,j] × X[j]

GRF_B[0] = Z A[182,j] × X[j]

GRF_B[0] = Z A[183,j] × X[j]

GRF_B[0] = Z A[184,j] × X[j]

GRF_B[0] = Z A[185,j] × X[j]

GRF_B[0] = Z A[186,j] × X[j]

GRF_B[0] = Z A[187,j] × X[j]

GRF_B[0] = Z A[188,j] × X[j]

GRF_B[0] = Z A[189,j] × X[j]

GRF_B[0] = Z A[190,j] × X[j]

GRF_B[0] = Z A[191,j] × X[j]

GRF_B[0] = Z A[192,j] × X[j]

GRF_B[0] = Z A[193,j] × X[j]

GRF_B[0] = Z A[194,j] × X[j]

GRF_B[0] = Z A[195,j] × X[j]

GRF_B[0] = Z A[196,j] × X[j]

GRF_B[0] = Z A[197,j] × X[j]

GRF_B[0] = Z A[198,j] × X[j]

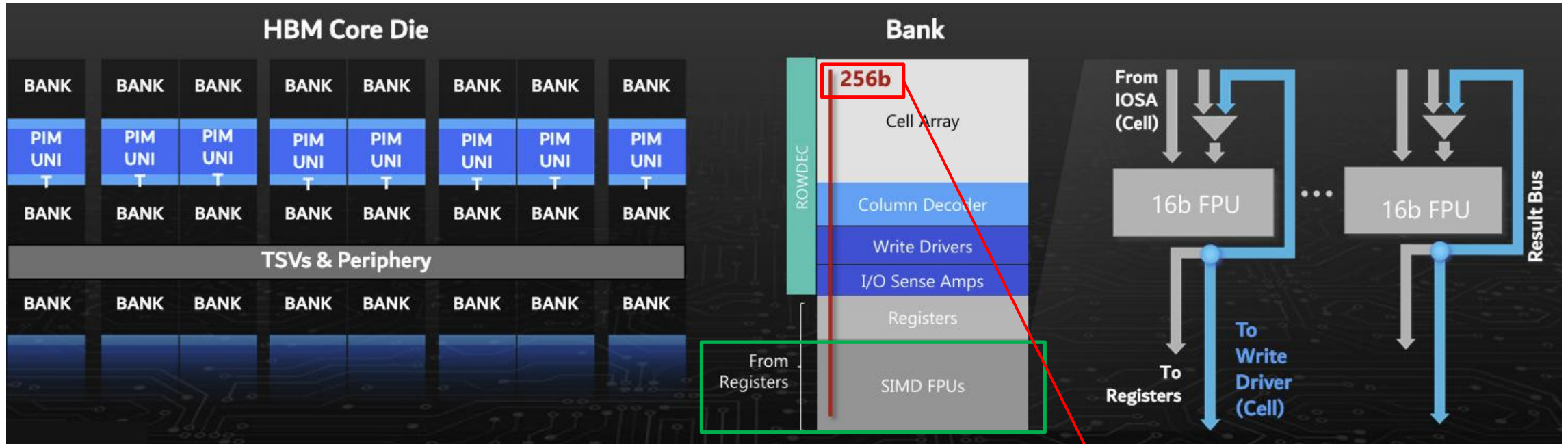
Samsung HBM-PIM Hardware

□ HBM-PIM Chip

• HBM-PIM Die

- Bank들이 병렬적으로 Processing가능 : access multi banks/PCUs in a **lockstep manner(= synchronized)**
- 1개의 Channel 내에서 모든 Bank(=16개)는 동기화되어 PIM연산 진행
- Support both standard HBM and Aquabolt-XL modes for versatility

```
NUM_BANK_GROUPS=4  
NUM_BANKS=16  
NUM_COLS=128  
NUM_ROWS=16384  
NUM_PIM_BLOCKS=8  
DEVICE_WIDTH=64  
BL=4
```



16개의 SIMD FP16 Mul & Add units

HBM2: Pseudo channel 당 256bit/transaction

Samsung HBM-PIM Hardware

❑ HBM-PIM Chip

- HBM-PIM Die
 - 연산 능력은 아래와 같이 파악됨 (Paper 기준)
 - Simulator에서는 Pseudo-Channel = 64로 하여 Simulation을 진행함

NUM_BANK_GROUPS=4
NUM_BANKS=16
NUM_COLS=128
NUM_ROWS=16384
NUM_PIM_BLOCKS=8
DEVICE_WIDTH=64
BL=4

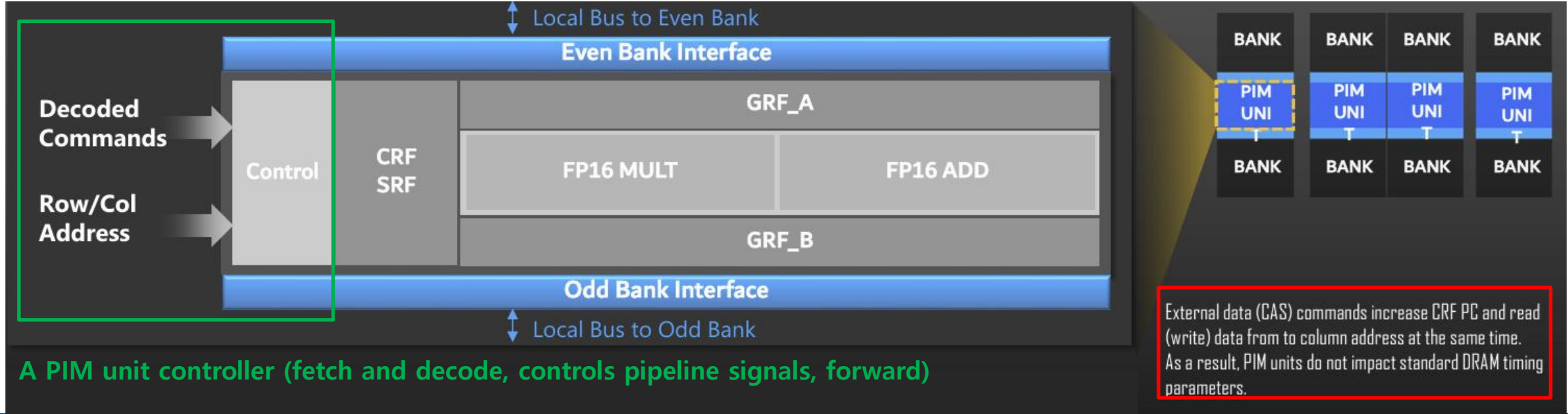
계층	단위	병렬 처리 단위 수	처리 Data 양 (FP16 기준)	처리량 per Burst
1. PU 1개	SIMD 연산	16개 FP16 PCU	16 x 16-bit	256 bits
2. Pseudo-Channel 1개	PIM Block (PCU)	8개	8 x 256-bit	2,048 bits (256 Bytes)
3. HBM-PIM Device 1개	Pseudo-Channel	16개 (paper) 64개 (simulator)	16 x 2,048-bit 64 x 2,048-bit	32,768 bits (4,096 Bytes) 131,072 bits (16384 Bytes)

Samsung HBM-PIM Hardware

□ HBM-PIM Chip

일반 동작: 64bit x 4burst

- HBM-PIM Processing Unit : PU 당 16 x (FP16 MULT & Adder) → PIM mode: Bank에서 load시 일반 동작과 다르게 한번에 256bit load
 - 5 stage Pipeline을 가짐 → DRAM Internal timing for RD/WR을 만족시키기 위하여
 - Fetch & Decode → Load Data → MULT → ADD → Write Back (명령어 종류에 따라 특정 stage Skip 가능)
 - PIM 연산은 기존 Read/Write Transaction의 Timing을 그대로 따름 → Pipeline 구조로 연산하기에 가능한 것으로 보임
 - CAS commands (= RD/WR)가 CRF PC값을 올려서 PIM command를 실행
 - PIM mode 시 PIM 연산을 위한 local path(= independent path)를 사용하기에 DRAM timing 영향 X



Samsung HBM-PIM Hardware

□ HBM-PIM Chip

- HBM-PIM ISA (32bit)
 - Three Instruction types (Total 9 instructions)
 - 3 Control flow: NOP, JUMP, EXIT (JUMP: Zero-cycle static branch → supports only a **preprogrammed numbers of iterations**)
 - 2 Data transfer: MOV, FILL
 - 4 Arithmetic: ADD, MUL, MAC, MAD

Type	Operation	Operand (SRC0)	Operand (SRC1)	Result (DST)	# of combinations
Arithmetic	ADD	GRF, BANK, SRF	GRF, BANK, SRF	GRF	40
	MUL	GRF, BANK	GRF, BANK, SRF	GRF	32
	MAC	GRF, BANK	GRF, BANK, SRF	GRF_B	14
	MAD	GRF, BANK	GRF, BANK, SRF	GRF	28
Data Transfer	MOVE	GRF, BANK		GRF, SRF	24
	FILL	GRF, BANK		GRF, BANK	12
Control Flow	NOP				1
	JUMP				1
	EXIT				1

Samsung HBM-PIM Hardware → Simulator CMDs

❑ PIM Instruction Set Architecture

- Simulator의 PIM CMDs (Same with HBM-PIM paper)

Type	Command	Description	Result (DST)	Operand (SRC0)	Operand (SRC1)
Arithmetic	ADD	Addition	GRF	GRF, BANK, SRF	GRF, BANK, SRF
Arithmetic	MUL	Multiplication	GRF	GRF, BANK	GRF, BANK, SRF
Arithmetic	MAC	Multiply-Accumulate	GRF_B	GRF, BANK	GRF, BANK, SRF
Arithmetic	MAD	Multiply-Add	GRF	GRF, BANK	GRF, BANK, SRF
Data	MOV	load/store data from register to bank	GRF	GRF, SRF	GRF, BANK
Data	FILL	copy data from bank to register	GRF	GRF, BANK	GRF, BANK
Control	NOP	do nothing	-	-	-
Control	JUMP	jump instruction	-	-	-
Control	EXIT	exit instruction	-	-	-

Samsung HBM-PIM Hardware

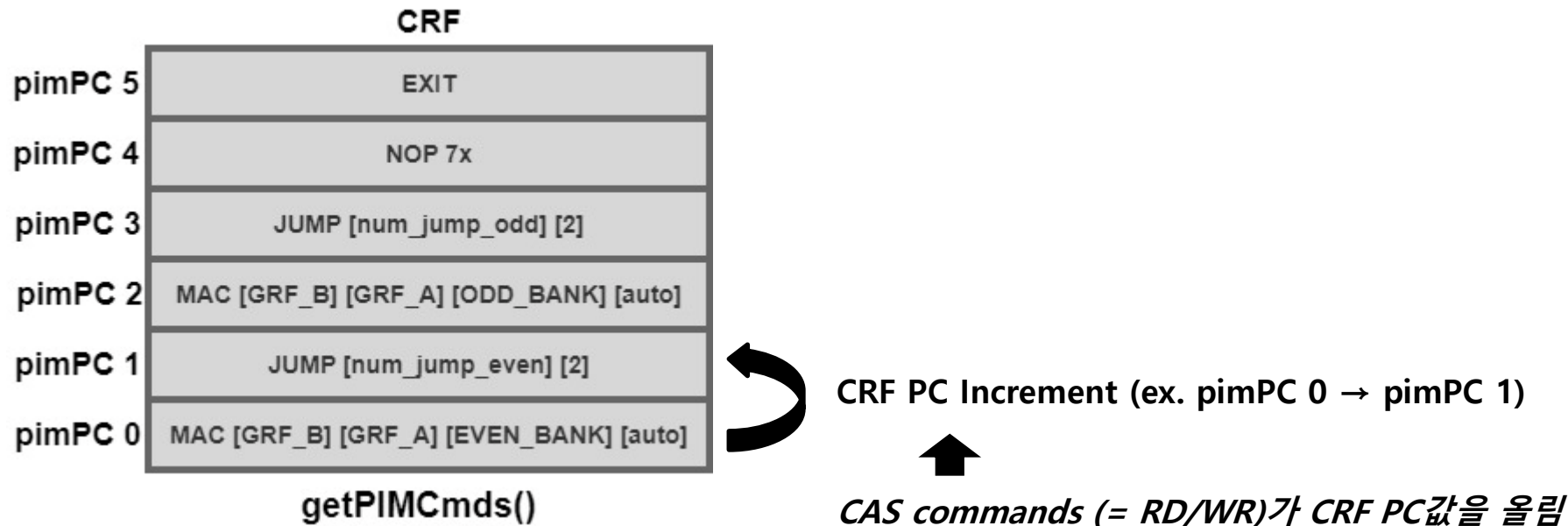
❑ PIM Instruction Set Architecture

- RISC-style 32비트 Instructions를 Support
- 3가지 Instruction 유형:
 - 4개의 Arithmetic: ADD, MUL, MAC, MAD
 - 2개의 Data Transfer: MOV, FILL
 - 3개의 Control: NOP, JUMP, EXIT
- JUMP Instruction: 0-cycle static branch
 - 미리 programming된 반복 횟수만 지원
- Operand 유형:
 - Vector Register(GRF_A, GRF_B)
 - Scalar Register(SRF)
 - Bank Row Buffer

Samsung HBM-PIM Hardware

❑ PIM Instruction Set Architecture

- PIM Instructions는 CRF에 저장됨, Memory Instructions가 CRF를 trigger하여 target instruction를 수행
 - 각 Memory Instruction이 CRF PC를 increment
- DRAM Instruction이 PIM 연산을 위해 DRAM에서 data를 가져오는 위치를 결정
 - 참고 : PIMSimulator의 readOPd() 함수



Samsung HBM-PIM Hardware

❑ HBM-PIM Chip

- HBM-PIM ISA (32bit)

- R**: ReLU 켜기 Switch (MOV Command와 함께 사용)
 - MOV(ReLU)라고 쓰면 Data 이동하면서 음수 값을 0으로 만듦
- U**: 사용 안 하는 bit (여유 공간)
- A**: Arithmetic Operation에서 Adress Align Mode(AAM) 사용 표시
 - AAM은 Memory Controller에 의해 Command들이 Reorder 되어도 올바른 MAC연산 순서를 보장하기 위해 사용

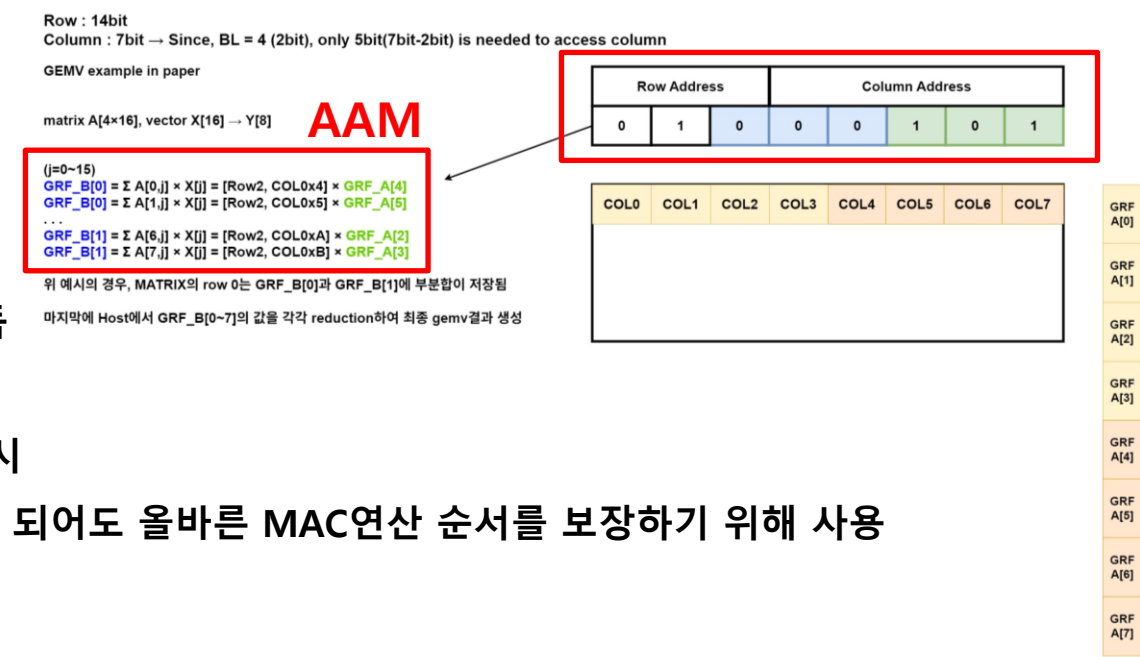


TABLE III: The PIM-HBM instruction format.

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Control	OPCODE				U								IMM0								IMM1											
Data	OPCODE				DST			SRC0			U								R	U	DST #		U	SRC0 #		U	SRC1 #					
ALU	OPCODE				DST			SRC0			SRC1			SRC2			A	U		U	DST #		U	SRC0 #		U	SRC1 #					

Samsung HBM-PIM Hardware

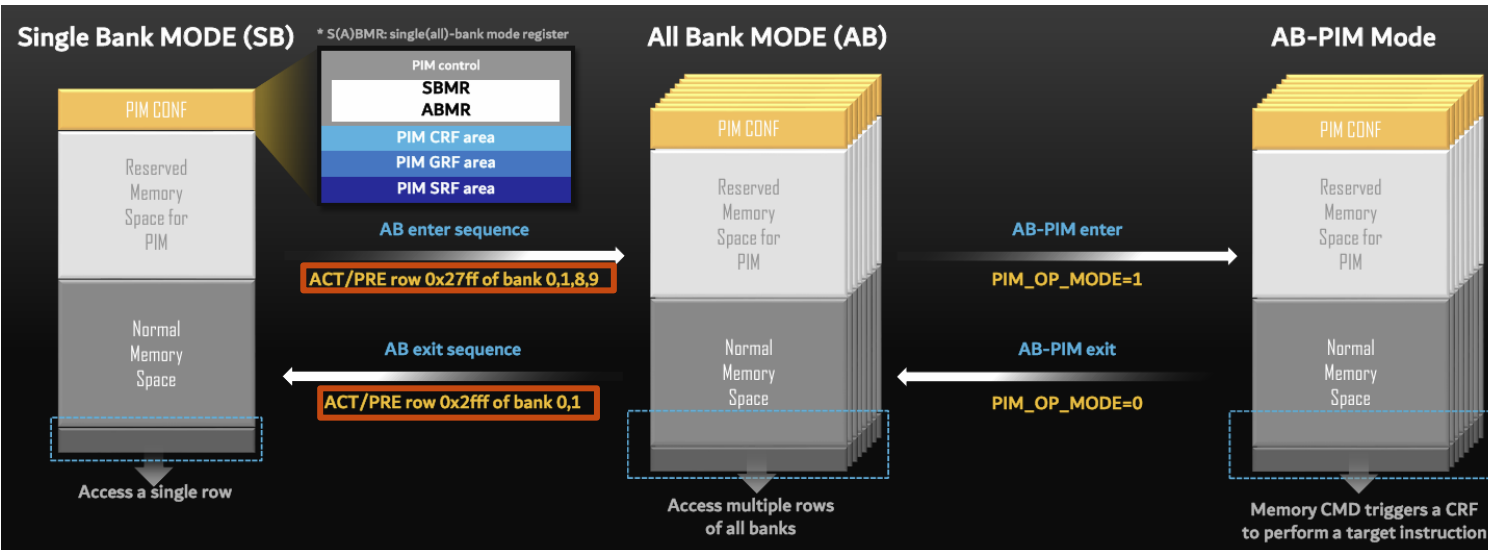
□ HBM-PIM Chip

• HBM-PIM Mode Transition

- PIM_CONF라는 예약된 Memory 공간에 2개의 Register를 Mapping
- AB Mode Enter: Host가 PIM_CONF의 특정 Address에 ACT와 PRE명령을 순서대로 보냄 → Simulator에서는 ACT만 해도, AB mode
- AB Mode Exit: Host가 PIM_CONF의 특정 Address에 PRE명령을 보냄
- AB-PIM Enter/Exit: WRITE Command를 통해서 접근 후 PIM_OP_MODE (= packet data영역의 특정 bit)를 확인

```
PIM_REG_RA = 0x3fff;  
PIM_ABMR_RA = 0x27ff;  
PIM_SBMR_RA = 0x2fff;
```

Configuration.h



- Simulator에서는 Mode Transition 오버헤드를 Modeling 안 함
- Example
 - WRITE command 시 PIM_OP_MODE 제어 data를 Data Bus로 보내야 할 텐데, Command Bus의 WRITE를 받자마자 data를 보고 바로 mode transition
 - 분명, Command Bus의 WRITE는 data를 가지지 않을 것임
 - 즉, WRITE command만 받으면 0 cycle만에 mode 바뀜

Samsung HBM-PIM Hardware

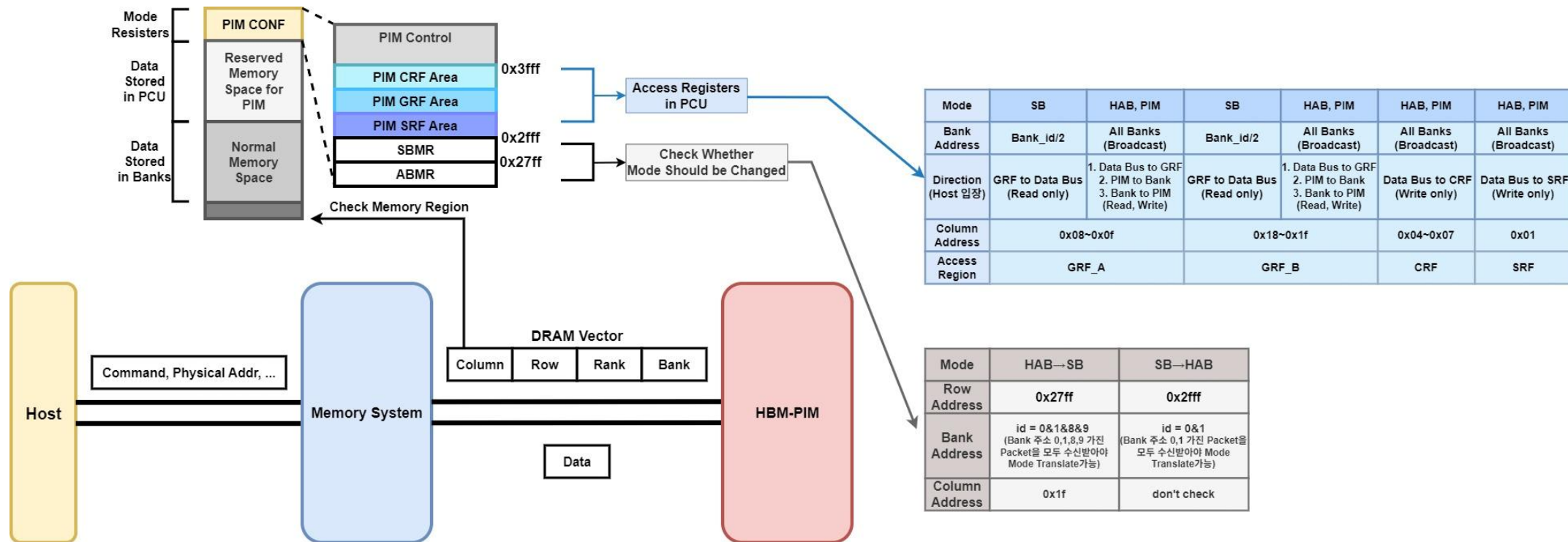
□ HBM-PIM Chip

• HBM-PIM Mode Transition

- 각 PIM_Mode에서 Registers Area 접근 시 Data Transfer되는 양상
- 각 PIM_Mode가 Transition되는 조건 (SB↔HAB↔PIM)

```
PIM_REG_RA = 0x3fff;
PIM_ABMR_RA = 0x27ff;
PIM_SBMR_RA = 0x2fff;
```

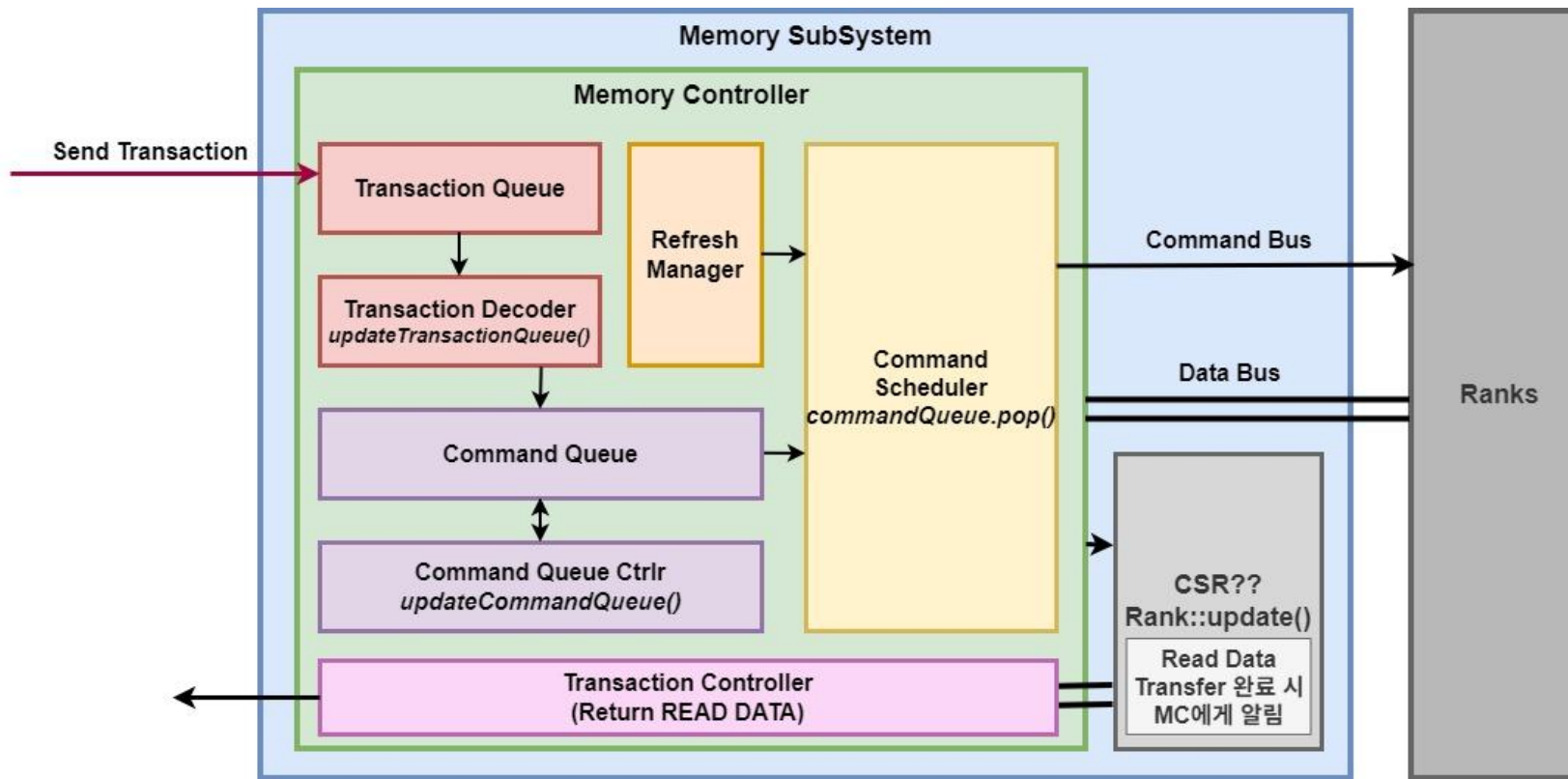
Configuration.h



Samsung HBM-PIM Hardware

□ HBM-PIM Chip

- HBM-PIM Memory System



- Memory Subsystem 역할

- Initialization
- Address/Command Translation
- Host의 read/write 요청 → DRAM Vector 변환
- Read Data 받을 시, Host에게 알림
- Command Scheduling
- DDR Timing Constraints 관리

Samsung HBM-PIM Hardware

❑ Movement of Data

- **SB mode: standard DRAM operation**
- **HAB mode: Allowing concurrent accesses to multiple banks with a single DRAM command**
 - Host → SRF write (256bit)
 - Host → CRF write (256bit) / pimPC 증가할 때 마다 32bit씩 내부적으로 Instruction을 Read
 - GRF data → Bank read/write
- **PIM mode: Triggers the execution of PIM instructions on the CRF by DRAM Command**
 - 256bit Data를 [GRF or SRF or Even Bank or Odd Bank]에서 Read/Write

Mode	Transaction	PIM Instruction	Operation
SB	Read	-	Normal Memory Read
SB	Write	-	Normal Memory Write
HAB	Write	-	PIM Write (Host to PIM Register)
PIM	-	MOV	read or write from bank to PIM Register

Samsung HBM-PIM gemv Operation

□ gemv Test Scenario

- 앞선 Hardware 설명과 같이, HBM-PIM은 **Lock Step**으로 operation함
 - 즉, 모든 PIM Block의 CRF에는 같은 Program이 write되며,
각 Channel내의 모든 PIM Block의 동작은 synchronize되어 수행
- FP16 Data
- $(4096 \times 1024 \text{ Weight Matrix}) \times (1024 \times 1 \text{ Input Vector}) = (4096 \times 1 \text{ Output Vector})$

Samsung HBM-PIM gemv Operation

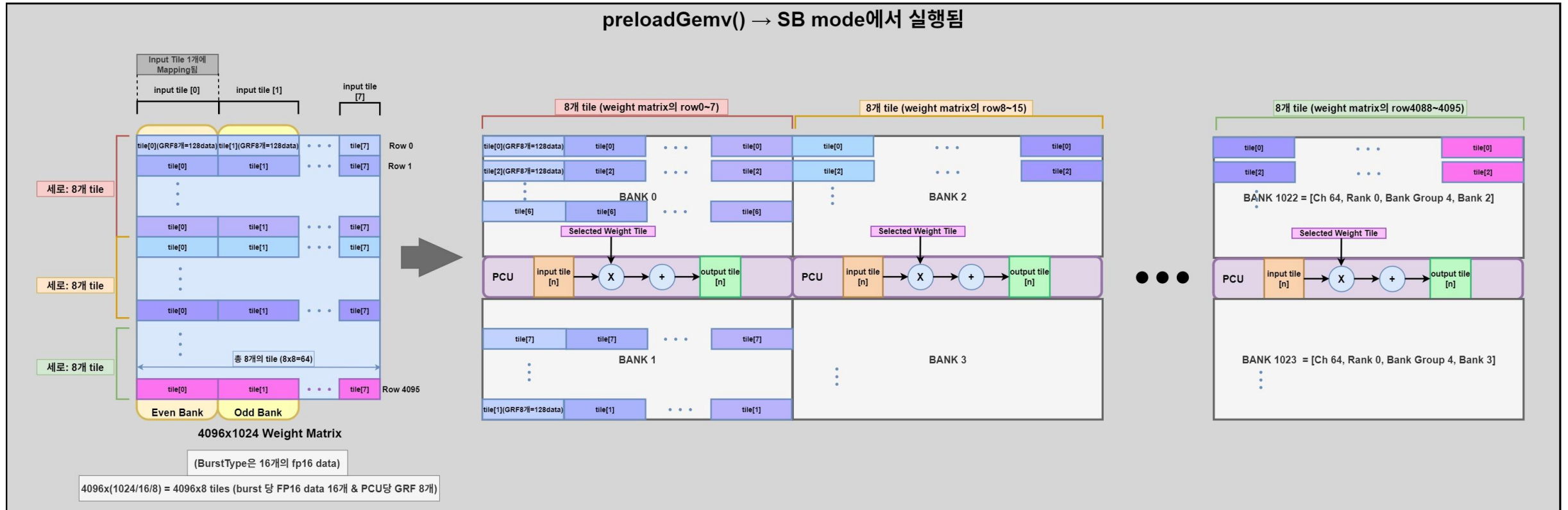
□ gemv Test Scenario – Tiling 단위 분석

- Gemv operation은 Hardware의 Capacity를 고려한 **Tiling**을 통해서 진행된다.
 - **Input Vector** → PCU안의 GRF_A에 Broadcast되어 저장됨
 - 총 capacity = 8 GRF x 256bit = 8 GRF x 16 x FP16 = 128 FP16 data
 - 1024 x 1 Input Vector → **8개의 Input Tile** 필요 (Tile Size = 128 FP16 data)
 - **Weight Matrix** → HBM의 Bank에 저장됨 (아래, 256bit/16bit = 1 Burst / FP16단위)
 - 64channel x 16banks = **1024 Banks**에 Distribute됨 → 2개(Even/Odd pair)의 Bank당 8x1024 FP16 data
 - **Even Bank**에는 Weight Matrix의 **짝수 Column**, **Odd Bank**에는 Weight Matrix의 **홀수 Column** 저장
 - 다음 Page에 어떻게 Weight가 Distribute 되는지 설명
 - **Output Vector** → PCU안의 GRF_B에 저장, 1개의 연산 단위가 끝나면, GRF_B의 연산완료된 data를 Bank로 Back Up
 - 총 capacity = 64channel x 8PCU x 8 GRF x 256bit = 64 x 8 x 8 x 16 x FP16 = 4096 x 16 FP16 data
 - 4096 x 1 Output Vector → **1개의 Output Tile** 필요 (Tile Size = **4096 x 16 FP16 data**)
 - 4096 x 16 FP16 data는 4096 x 1 Output Vector의 Partial Sum
 - Host에서 4096 x 16 FP16 data를 **Reduction**하여 4096 x 1 Output Vector로 만들

Samsung HBM-PIM gemv Operation

□ Weight Matrix Distribution – preloadGemv()

- SB mode에서 수행됨
 - 4096 x 1024 weight matrix를 모든 channel의 Bank들에 분배 (아래 그림과 같이 분배됨)



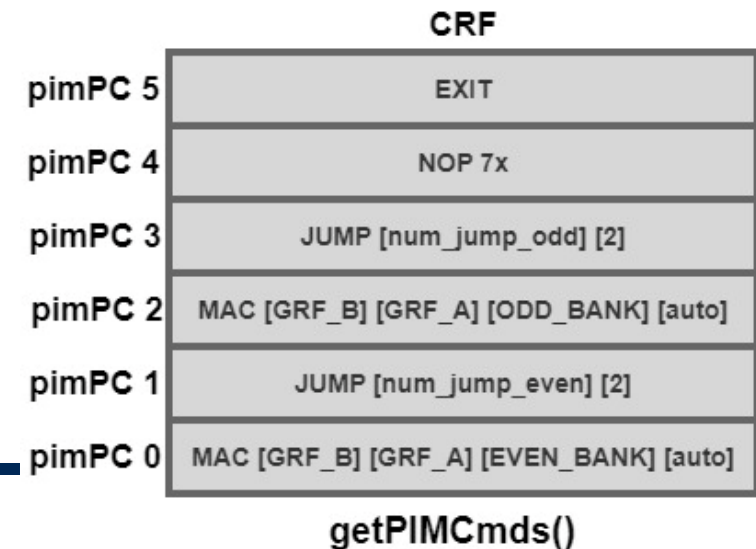
Samsung HBM-PIM gemv Operation

❑ Input Vector Transfer & Compute – executeGemv()

Weight Matrix를 Bank에 Distribute 후,

0. CRF를 프로그래밍 함.

- gemv에 해당하는 Instruction을 CRF에 write함
- getPIMCmds() 함수를 통해서 Instruction data를 가져옴
 - gemv 연산에 대해서 num_jump_even, num_jump_odd만 받으면 미리 정의된 CMDs를 반환함
 - 각 Bank에 8개의 weight tile[n]이 있으며, INPUT TILE은 총 8개(even/odd 각각 4개)임.
 - 따라서, num_jump_even/odd는 각각 $8 \times 4 - 1 = 31$ 임 (0~31까지 총 32번 auto MAC cmd가 JUMP에 의해 반복 실행)
- programCrf()를 통해서 CRF에 접근하는 Reqeust(=Transaction)을 생성
 - CRF에 getPIMCmds()를 통해 가져온 Instruction들을 write

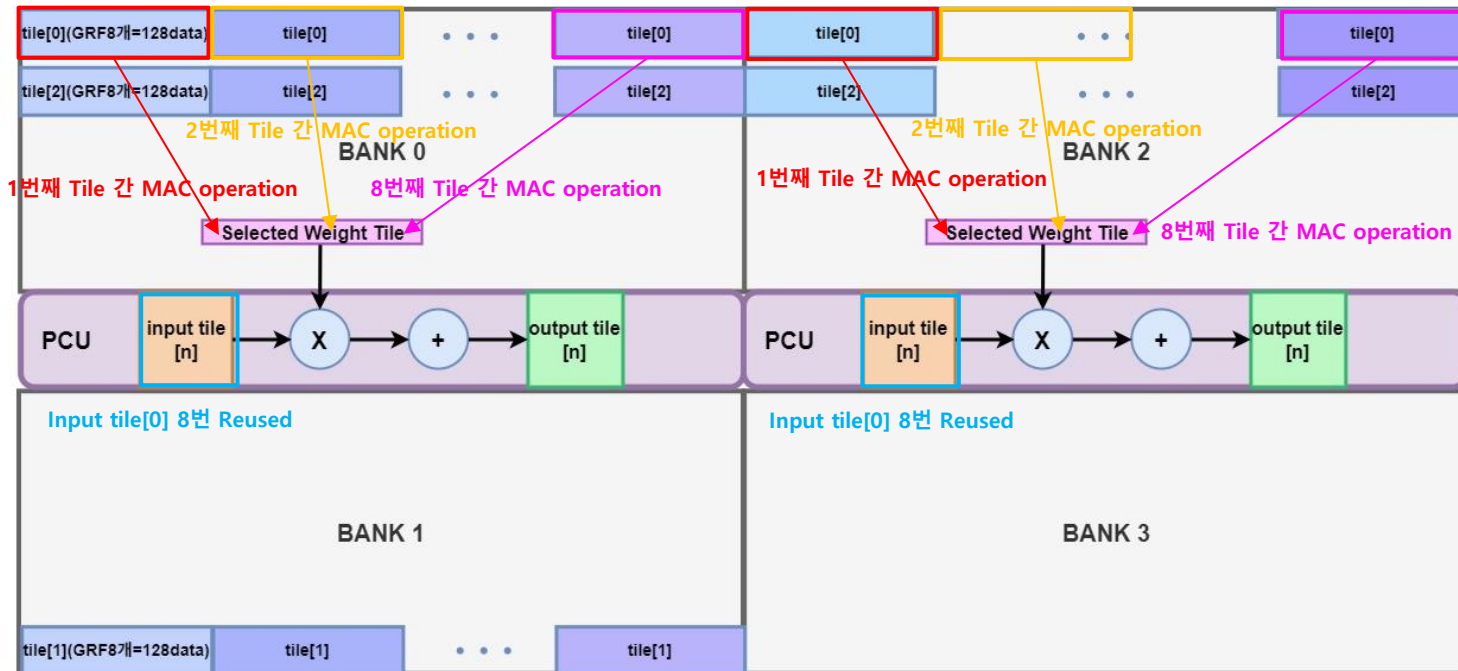


Samsung HBM-PIM gemv Operation

□ Input Vector Transfer & Compute – executeGemv()

Weight Matrix를 Bank에 Distribute 후,

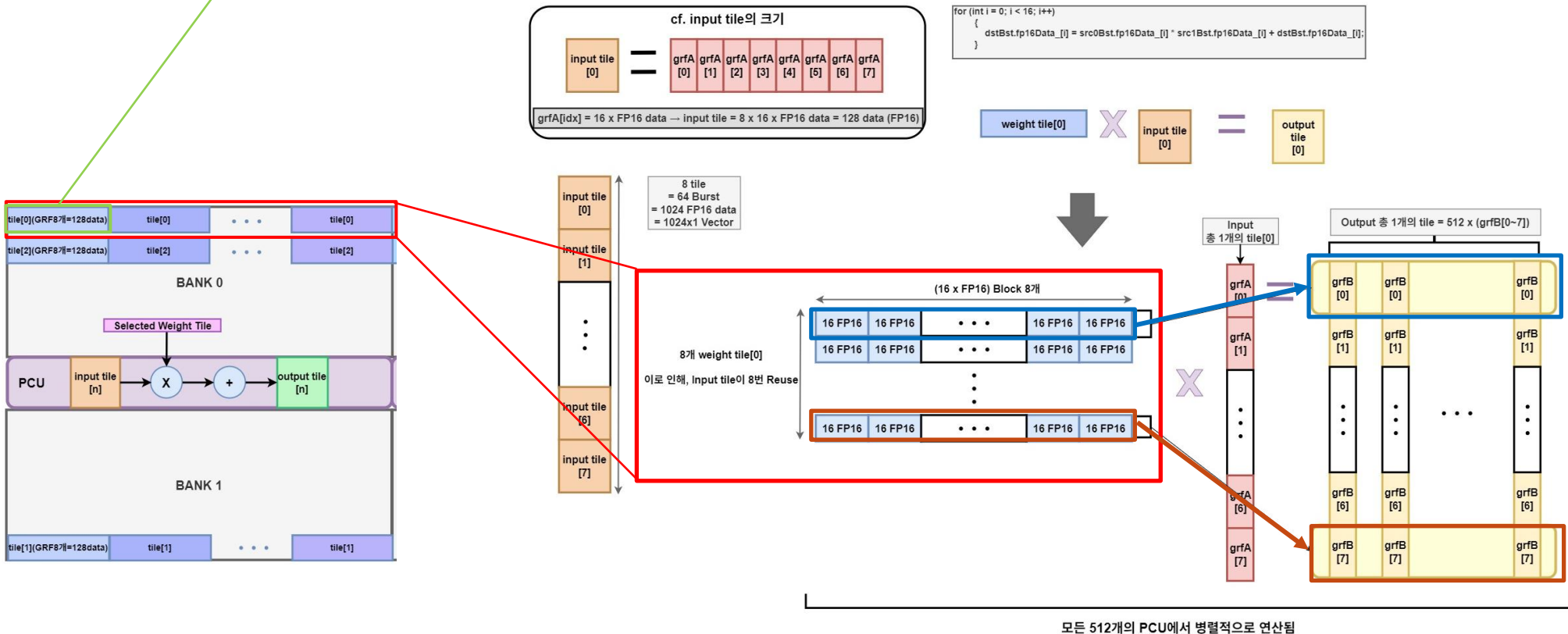
1. Host에서 Input Vector Tile을 PIM Block으로 보내 GRF_A에 Write
 2. GRF_A에 저장된 Input을 최대한 Reuse(=8번)하는 방향으로 Bank에 저장된 Weight와 MAC연산을 수행
- 위 1, 2 step은 HAB/PIM mode에서 수행됨



Samsung HBM-PIM gemv Operation

❑ Input Vector Transfer & Compute – executeGemv()

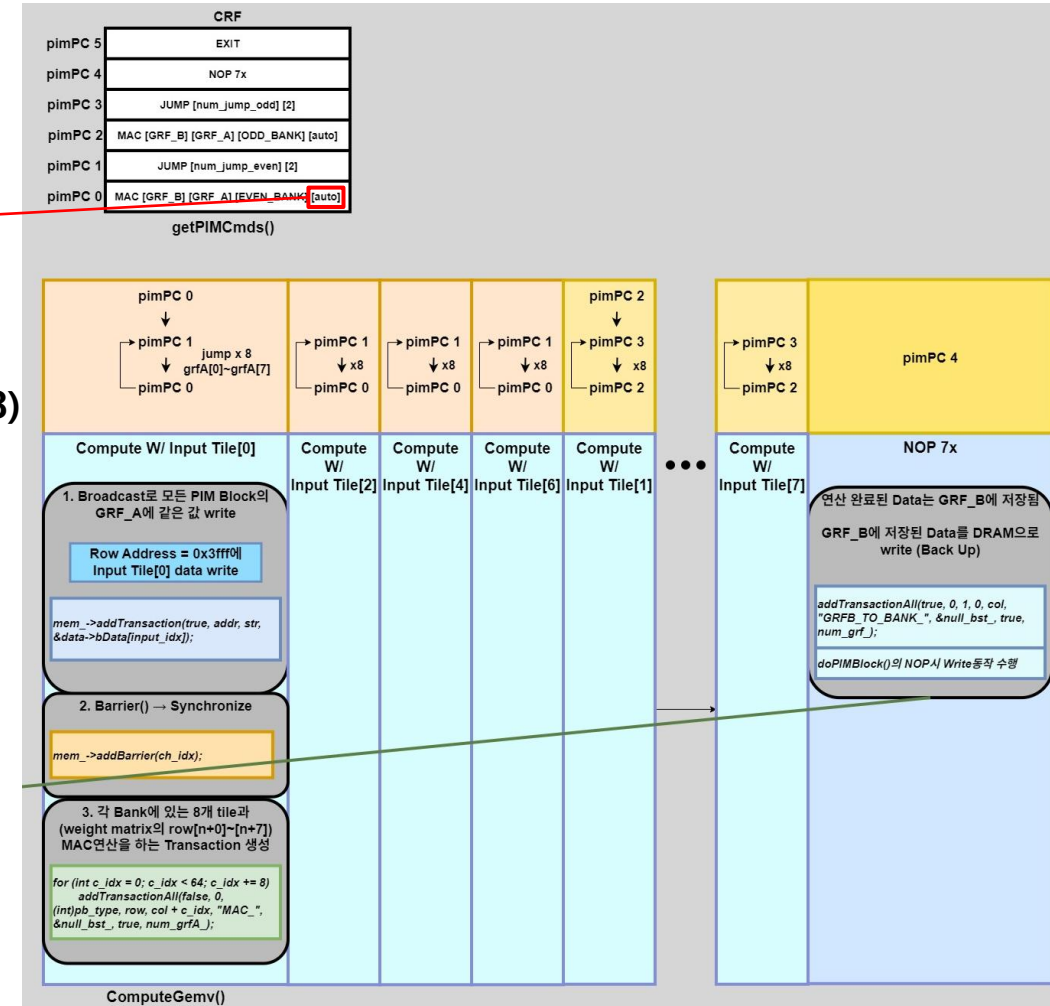
- Weight tile[n]의 1개의 segment (256bit x 8) 와 Input tile[n]간 16 way SIMD MAC 연산 수행
- grfA[0~7]과 Weight tile[n]간 MAC연산 시, grfB[0~7]에 저장됨



Samsung HBM-PIM gemv Operation

□ Input Vector Transfer & Compute – executeGemv()

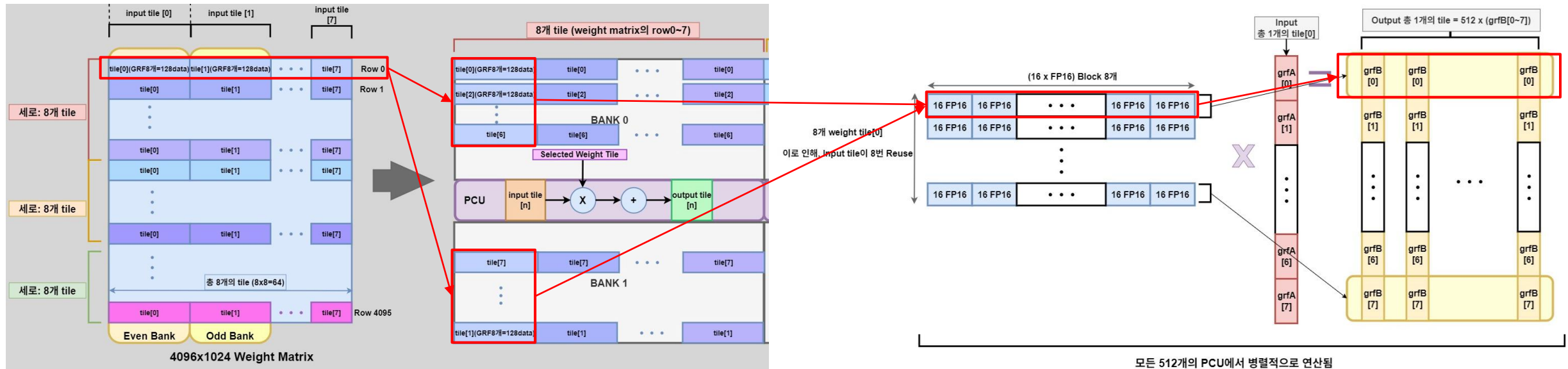
- executeGemv() - CRF의 PC값에 따른 execution 흐름
 - CRF는 오른쪽과 같은 Instruction을 가지고 있음
 - 0번 INPUT Tile을 load 하고, Even Bank의 weight Tile을 접근
 - pimPC0에서 MAC이 실행되면 16 SIMD FP16 MAC을 수행
 - auto = true이기에 8번 간 Read command마다 pimPC0(=MAC) 수행
또한, Bank에 존재하는 8개의 Weight tile에 대해서 MAC 수행 (jumpx8)
 - 다음 INPUT Tile(=2번 INPUT Tile → Even Tile)을 load
 - pimPC1에서 JUMP로 인해 pimPC0이 되고, 3.~4. 반복
 - 다음 INPUT Tile을 load 후, JUMP 되고, 3.~4. 반복
 - ... 4개의 INPUT Tile을 load 및 MAC연산 후, pimPC=2가 됨
 - 이제 1번 INPUT Tile을 load 하고, Odd Bank의 weight Tile을 접근
 - ... 3.~4. 반복
 - pimPC4에서 NOP 7x를 수행
→ 이때, GRF_B에 accumulate된 data를 Bank로 옮김



Samsung HBM-PIM gemv Operation

□ Input Vector Transfer & Compute – executeGemv()

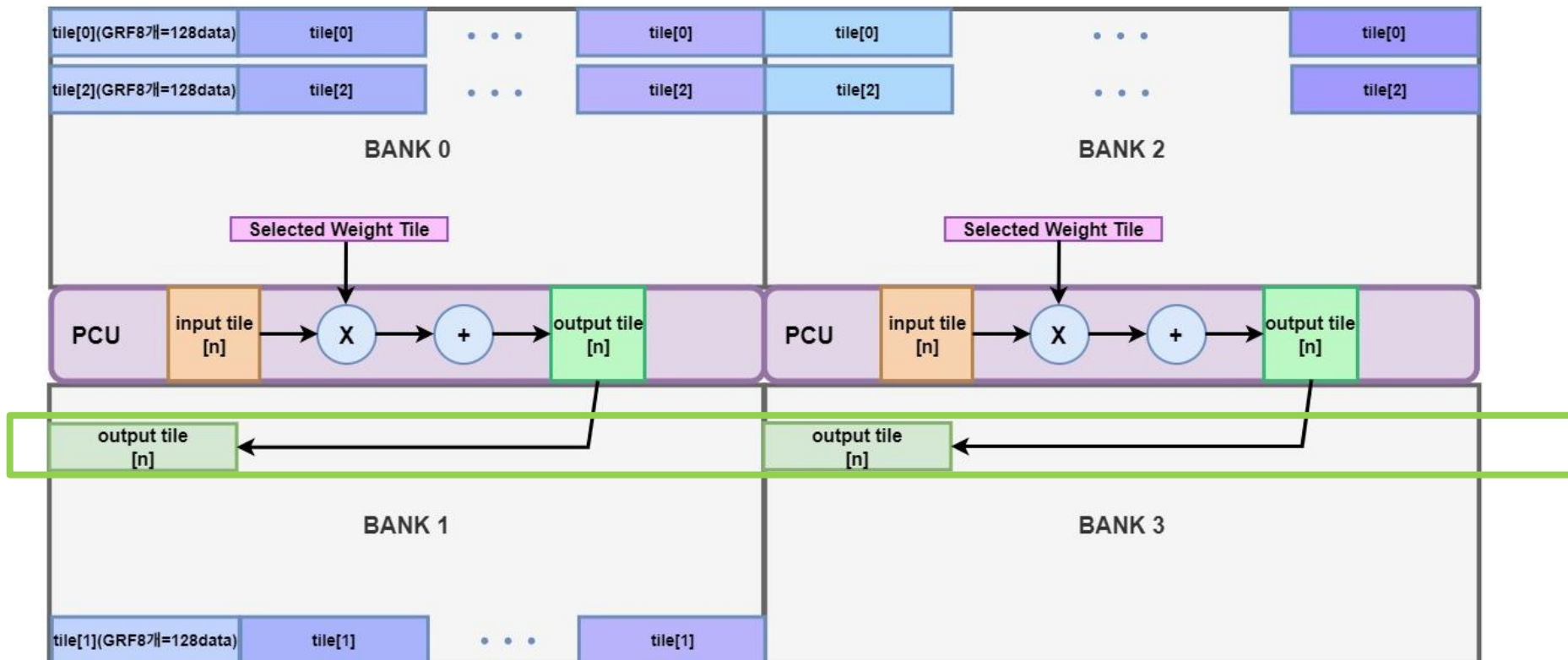
- Output Vector가 GRF_B에 저장되는 형태
 - Weight Matrix의 0번 Row에 대한 vector x vector 결과는 BANK[0&1]의 GRF_B[0]에 저장됨
 - Weight Matrix의 1번 Row에 대한 vector x vector 결과는 BANK[0&1]의 GRF_B[1]에 저장됨
 - Weight Matrix의 4095번 Row에 대한 vector x vector 결과는 BANK[1022&1023]의 GRF_B[7]에 저장됨



Samsung HBM-PIM gemv Operation

□ Input Vector Transfer & Compute – executeGemv()

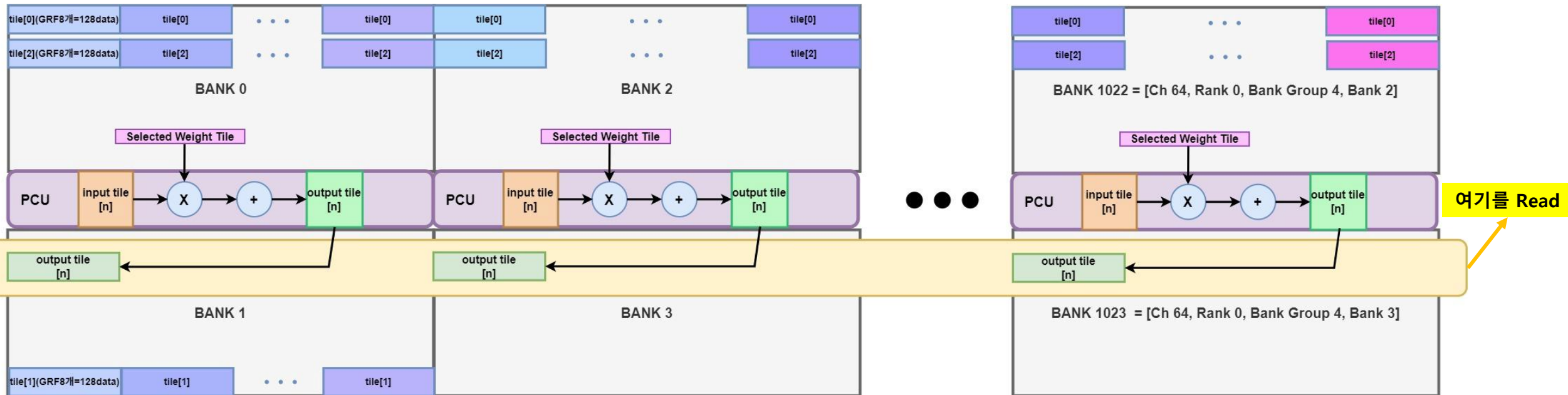
- 1번의 연산단위(= 처음에 설정한 Input Tile 8개에 대한 연산)가 끝나면, (→ pimPC4 = NOP 7x 동안)
- GRF_B에 accumulate된 data를 Bank로 옮김.



Samsung HBM-PIM gemv Operation

❑ Output Vector Read – readResult()

- SB mode에서 수행됨
- 차례로 Odd Bank에 저장되어 있는 4096 burst의 연산 결과(= 4096x16) Data를 가져옴
- 나중에 Host 쪽에서 burst별로 Reduction Sum을 진행하여 4096x1 Vector를 결과로 출력
 - 즉, 4096x16 → 4096x1



Samsung HBM-PIM gemv Operation

❑ Output Vector Read – readResult()

- readResult() 총정리

